Temco Platform

Getting started · built Nov 12, 2026

Getting started

Temco Platform is where product documentation is written and published. It is a desktop application, built on VS Code. In it you write pages and group them into a **project**. You then build that project into a website, with a printable PDF and a Windows help file, and publish the result to **Temco Learn**: the documentation site readers actually open, at `https://temcocontrols.github.io/temco-learn/`.

This guide walks that whole journey once, from an empty folder to a live page. It assumes you have never opened the application before, and it assumes no background in VS Code, in Git, or in the site's own code. Where one of those cannot be avoided — a commit, a catalog — it is explained at the step where you meet it.

The 30-second version

- **What it is:** you write pages in a project, build them into a site, and publish that site to Temco Learn.
- **What it needs:** the application, Node 22, a clone of the site with its dependencies installed, and permission to push to `temcocontrols/temco-learn`.
- **What you run:** `temcoDocs.buildSite`, then `temcoLearn.addBuild`, then `temcoLearn.site.downloads`, then `temcoLearn.publish`.
- **What you get:** your project on `temcocontrols.github.io/temco-learn/`, under Documentation, with one version for each build you add.

Read steps 1–9 the first time through. After that, everything you need day to day is in Running it again.

> **Screenshot — the finished result: a project card under Documentation, opened to a page, with the version selector and the download menu visible.** `media/publishing/00-published-page.png`

The two halves

Temco Platform and Temco Learn are two halves of one idea, and they meet in a single place: a copy of a project's build.

```text
you, writing          Temco Platform              the site clone            Temco Learn
a page          →      builds and publishes  →     one commit on main   →    what a read
```

	Temco Platform	Temco Learn
is	the **authoring** application, a desktop app built on VS Code	the **published** site, a static React app on GitHub Pages
is for	whoever writes, reviews and owns product documentation	whoever reads it: customers, field engineers, support
what you do there	create a project, write pages, build, make the downloads, publish	read, search, switch versions, download the PDF or the help file
what it holds	your project folder, and the site clone it publishes into	one imported build per version, the catalog, the downloads
where it lives	the application on your machine	its own repository, `~/temco/temco-learn`, deployed by CI

You only need the left-hand column to finish this guide. The right-hand one is what gives the work its point, and it is also the reason there is a step that produces a PDF by hand: the site cannot make one for you.

What you see when you open Temco Platform

The activity bar on the left holds eight areas. This guide uses four of them, and it is worth knowing what the other four are before they surprise you.

Area	Holds	In this guide
Home	Overview, Projects, What needs doing	step 2: where a project is created
Documentation	Project, Pages, Publish	steps 3 and 4: the pages, and the build
Repositories	Sources: organisations, clones, the folder settings	step 1: the folders everything lands in
Temco Learn	Overview, Site, Artifacts	steps 6 to 9: add, downloads, preview, publish
AI	Draft, What it does, Server, Tools	Beyond the flow: drafts beside your text
Forum	Browse	not covered here
Tools	Studios: SQL, network, BACnet, devices	not covered here
Settings	General, Modules, AI Runtime, Branding, Publishing	step 1, and the Repositories page above

If you cannot find something in the activity bar, you can still reach it by name: the appendix lists every control together with the command id behind it.

What a reader sees on Temco Learn

It is worth seeing this once, so that you know what you are aiming at.

The reader opens	And gets
the home page	the site's sections as tiles; Documentation is where your project will appear
a section	the projects filed under it, each with its title and summary
a project	its versions — every version you add is a separate entry, never an overwrite
a version	that version's pages, in the reading order the registry gives them
a page	the page itself, a download menu for the PDF, the help file and the Markdown, and a way to say it was wrong

None of this is a separate publishing channel. The site reads one catalog, and that catalog is what step 6 writes.

Who this is for

If you are…	this guide gets you to
writing documentation for a Temco product (T3000, a controller, a protocol)	a project of your own, built and live under Documentation on the site
taking over documentation that already exists — a folder of Markdown, a Word manual, a compiled `.chm`	the same result, starting from *Import a document set* in step 2, with your existing pages converted once
working inside a product repository that already has `docs/` and its own build scripts	the same result, with your repository's own `docs:site` or `build:help:dry` scripts doing the building instead of the built-in generator
reviewing someone else's documentation	enough of the flow to read what they changed, and to publish it

You do not have to be the person who set the machine up. If somebody else did, Before the first time shows you how to check that the three settings and the site clone are right before you begin.

How to use this guide

- **Read steps 1–9 in order, once.** After that, go straight to Running it again; that is what everyday work looks like.
- **Every step has the same three parts:** what you do, **You should see** (a file or a screen to check), and **If not** (the usual cause). If the file a step promises is not there, that is the step that failed, and nothing after it is worth testing yet.
- **Buttons are named as they appear**, with the command id in backticks beside them, like `temcoDocs.buildSite`. If a button is hidden or unavailable, type that id into the Command Palette (`Ctrl+Shift+P`) and it will run the same action.
- **A `~` in a path means your home folder.** Windows paths are written with backslashes; on macOS and Linux the same path uses `/`. The only genuinely Windows-only piece is the `.chm`, which needs the help compiler.
- **Keep this guide open in a second window** while you work through it. It is meant to be read next to the application, not memorised.

What you need before you start

Need	Why
the application installed and running	this guide is about using it, not installing it
write access to the `temcocontrols/temco-learn` repository, or someone who has it	the last step pushes a commit there. Without access, everything before it still works and simply stops at *publish*
git configured with your GitHub account	a publish is a `git add`, a `commit` and a `push`, all under your name
the site clone on this machine	`~/temco/temco-learn` unless you choose otherwise. The Site page clones it for you
an hour for the first run	steps 2 and 7 account for most of it
rather less every time after	a repeat publish is four commands and about two minutes

In this guide

Section	Answers
The 30-second version	what this is, what you need, and the four commands that do it
The two halves	Temco Platform and Temco Learn, and where they meet
What you see when you open Temco Platform	the eight areas of the activity bar, and which four this guide uses
What a reader sees on Temco Learn	the site from the reader's side, so you know what you are aiming at
Who this is for	whether this guide is the right one for you
What you are making	what a project, a build and the site are, and where each one lives
The general flow	the five acts, what each writes, and what a reader notices
Words this guide uses	page id, area, chapter, version, section, catalog, guard, downloads
What you will have when you are done	the result, in five bullets
Before the first time	the setup that happens once per machine
Where things go	the three settings that decide every path
Steps 1–9	the flow, in order, each with what you should see
The editor	the ribbon, pictures, the outline and draft panel, preview, diff, and every shortcut
Running it again	the four commands for the second, third and hundredth publish
The same flow, as files	every file, and the step that wrote it
What the files look like	real snippets of the JSON you will meet
Reference: the outputs	the ticks, and which script each one runs
Two things worth doing on purpose	the tests that prove the guards really refuse
When a step goes wrong	symptom → cause → where to look
When something else goes wrong	the failures that are not about publishing
Beyond the flow	sources, the editor, validate, release, export, translate
Appendix	every button, and the command id behind it

What you are making

Five things take part. Knowing which is which is most of understanding the flow.

Thing	Is	Lives in
A project	one body of documentation: its name, its pages, and the outputs it should produce	`<projectsFolder>/<slug>/`, for example `~/temco/projects/t3000`
A page	one Markdown file, listed in the project's registry	`<project>/docs/<area>/*.md`, with the list in `docs/help/registry.json`
A picture	a screenshot or diagram a page points at — `![the trend log](media/x.png)`	`<project>/docs/media/`. The build copies everything under `docs/` that is not a page, so a picture keeps the address the page gave it
A build	the rendered site, made from those pages	`<project>/dist/site`, or `dist/docs` when your own generator wrote it
The site	the Temco Learn repository: a React app that reads a catalog of projects	`temcoLearn.siteRepo`, by default `~/temco/temco-learn`
The published page	what a reader opens on the internet	GitHub Pages, served from the site's `main` branch

The first three are yours and stay yours. The site is its own repository, and the last one belongs to GitHub. That split is why the flow has separate steps instead of one button, and why each step can fail in its own way.

The general flow

There are four acts and three places. You write and build inside **your project**. The site clone takes one version, plus its downloads. A publish puts that on **GitHub**, and the site's own CI serves it to readers.

TEXT

```
YOUR PROJECT              ──▶ THE SITE CLONE            ──▶ GITHUB, THEN READERS
write, then build             add, then the downloads       publish, then CI deploys
~/temco/projects/t3000        ~/temco/temco-learn           temcocontrols/temco-learn
```

Act	Command	Writes	What a reader would notice
write	the editor, `temcoDocs.newPage`	`docs/**/*.md`, `docs/help/registry.json`	nothing
build	`temcoDocs.buildSite`	`<project>/dist/site`	nothing yet: nobody can see it
add	`temcoLearn.addBuild`	`public/imported/<id>/<version>/`, `content/<id>/site.json`	nothing yet, though it previews at `localhost`
make the downloads	`temcoLearn.site.downloads`	`public/pdf/…`, `public/chm/…`, the download manifest	nothing yet, but the deploy would refuse without them
publish	`temcoLearn.publish`, then *Publish*	one commit, one push	a minute later, the page is live

The nine steps below are this table with the details filled in.

So the flow can stop in four places. Each is named at the step it belongs to: a build with no `index.html` to add, a site clone that is not where the setting says, a catalog that could not run, and a version whose PDF is missing. The last is the one people run into most often, which is why it has a step and a test of its own.

Words this guide uses

The platform uses a few words that mean something more specific than they do elsewhere. One pass over this table saves a lot of guessing later.

Word	Means
registry	`docs/help/registry.json`, the list of every page: its title, where it sits, whether it ships. The application reads it, and a build renders it
area	the top level of the documentation, one per product or section, declared in the registry
chapter	the folder a page sits in, inside an area
page id	the page's path under `docs/` without the `.md`: the id `t3000/commissioning/ports` is the file `docs/t3000/commissioning/ports.md`
build	the rendered site produced from the pages, in `<project>/dist/site`
version	one fixed entry of a project on the site, such as `1.0` or `1.1`. A rebuild becomes a new version rather than overwriting an old one. It becomes a folder and an address (`/t3000/1.0/`), so it is typed in digits — `1.0`, `17.4`, or a dated edition `2026.01` — never `v1.0`
section	the category a project appears under on the site: `documentation`, `training`, `tests`, `products`, `qa` or `topics`
catalog	`public/catalog.json` in the site, the one file the site's app reads to know what exists
downloads	the PDFs and help files the site serves. The deploy cannot build them, so they are made on your machine and committed
guard	the site's own check that every catalogued version has its downloads. CI runs it, and so does a publish before it commits

What you will have when you are done

- T3000 listed under **Documentation** on the site, with its own card, icon and description.
- Every version you added, each one selectable (`1.0`, `1.1`, `2.0`) with its own pages.
- A **Downloads** menu for each version: the whole-guide PDF, and a Windows help file when one could be compiled.
- Your pages in the site's search index.
- One commit on `main` in the site repository, and a green GitHub Actions run that deployed it.

Before the first time

The table above is about *you*: the application, access, an hour. This one is about the **machine**, and it happens once. If somebody else set the machine up, work down the list and confirm each line rather than assuming.

Do this once	Why
Node 22 or newer on your path (`node --version`)	the site's own scripts run with `--experimental-strip-types`
Have the site clone. The Site page (**Open the Site Page**) clones `temcocontrols/temco-learn` for you	the site is its own repository, and this application is a client of it
`cd ~/temco/temco-learn && npm ci`	the site's dependencies, once per clone
`npx playwright install chromium`	the PDFs are printed by a browser. Without it, *Make the Downloads* tells you so
Optionally, HTML Help Workshop	a `.chm` needs `hhc.exe`. Without it, the site simply has no help file

Three of those five are optional in practice: a site without PDFs still builds and previews, and a site without a help file is still a site. Node and `npm ci` are not optional. Without them the site's `catalog` cannot run, and step 6 will stop there.

Where things go

Three settings decide every path in this guide. Set them once. Every page reads them back, so you can check the answer rather than the field.

Setting	Where	Meaning
`temcoRepositories.workspaceFolder`	Settings ▸ Repositories ▸ Working folder	the working area. Clones go to `<it>/source/<repository>`
`temcoRepositories.projectsFolder`	Settings ▸ Repositories ▸ Projects folder	a project's own folder is made at `<it>/<slug>`
`temcoLearn.siteRepo`	the Site page ▸ **Choose the Site Folder**, or the setting	the site clone: `~/temco/temco-learn` unless you say otherwise

`temcoWorkbench.publishing.site` is not one of these. It is an address shown on the Publishing page, not a folder.

Step 1 — check the folders

Open **Settings ▸ Repositories** and read the two sentences under the fields, not the fields themselves. The page reads your saved settings, so those sentences are the honest answer to "where will this go?".

> **Screenshot — the Repositories settings page, with "Clones land in …" and "Projects are made in …" visible.** `media/publishing/01-folders.png`

You should see	`Clones land in …\temco\source` and `Projects are made in …\temco\projects`
If not	a value you typed but did not save still shows as the old one. Press *Choose* or leave the field, then look again. If it says `Projects are made in …\temco`, the projects folder is still empty, and new projects will land beside the clones

The third setting this guide depends on is **not** on that page. `temcoLearn.siteRepo` is the site clone; set it from the Site page with **Choose the Site Folder**. It defaults to `~/temco/temco-learn`. Check it now, because the whole right-hand half of the flow writes there.

Step 2 — create a project

Create a Project (`temcoProjects.newProject`). Give it the name `T3000` and the version `1.0`, and leave the outputs ticked: *Website*, *PDF*, *Offline help* and *Search index*.

> **Screenshot — the create-project form, with the output ticks visible and a name typed in.** `media/publishing/02-create-project.png`

Where the pages come from is the choice that decides what exists once the form closes:

Starting point	What it does
Start empty	records the project and nothing else. Its folder, `docs/help/registry.json` and one area come from **Create the Project's Documentation Folder** (`temcoDocs.createProjectFolder`)
From a template	writes a starter tree under the folder field — `docs/help/registry.json`, one area, two topics — named after the project, so `T3000` becomes `<projectsFolder>/t3000`
From a folder	cites a folder that already exists on this machine as the pages it reads; nothing is copied
From a repository	cites what the platform has already read — repositories, files and folders
Import a document set	converts Word, RTF, HTML, an HTML folder or a CHM **once** into `docs/` and a registry

Either way, the project is recorded under the slug of its name (`T3000` becomes `t3000`), the sources it cites are recorded with it, and the outputs become its recipe. No source is ever written to. A project is a

set of references, and its own folder holds only what the project itself writes.

You should see	`~/temco/projects/t3000/docs/help/registry.json`, and T3000 in the Library
If not	the folder is the slug of the name, so look in `~/temco/projects/` for what was actually made. A project created before you set the projects folder keeps the folder it already has; only new ones move

The folder is added to the window so that its `docs/` shows in Explorer. That is expected, and nothing needs doing about it.

Step 3 — write the pages

New Page (`temcoDocs.newPage`) with T3000 selected. It opens a **form** rather than a prompt: the place is already chosen from the row you were on, and the form shows the kind of page, the file it will write, and the body it starts from. The palette offers the kinds alongside it (*New Article*, *New Task*, *New Reference*, *New Troubleshooting*, *New Release Note*), and they differ only in their starter headings.

> **Screenshot — the Pages view and the New Page form side by side, with the kind, the target file and the starter body visible.** `media/publishing/03-pages.png`

In the form	Decides
the kind	the starter headings: *Task* gives steps, *Reference* gives a table, *Troubleshooting* gives symptom, cause and fix
area and chapter	where the file is written, as `docs/<area>/<chapter>/<title>.md`
the title	the page's name in the navigation, in search and in the tree. It is **not** the file name until you accept
which source	asked only when the project cites more than one folder

You are then in the editor. Each page is a Markdown file **and** a row in `docs/help/registry.json`. Three rules explain almost every surprise you will meet later:

Rule	What it means
the **page id is its path** under `docs/`, without `.md`	`docs/t3000/commissioning/ports.md` has the id `t3000/commissioning/ports`, and the registry must name it exactly that
`ship: false` keeps a page out of the built site	a page can exist and be edited, and still never appear
`order` decides the position inside a chapter	two pages sharing a title or an `order` are a warning, not an error

A page that is registered but whose file does not exist is an **error**, and the module reports it on refresh. That is what makes the `missing` rows in `Pages` mean something.

Everything else you will want on a page lives on its row. *Rename the Page Title*, *Move Up* and *Move Down* change where it sits; *Retire* keeps the file and drops it from the site; *Create the Missing Page File* fills a `missing` row; *New Folder…* (`temcoDocs.newChapter`) adds a chapter. Right-clicking a captured screenshot in `Needs You` offers *Attach the Screenshot to a Page…*.

You should see	a Markdown file under `~/temco/projects/t3000/docs/<area>/`, and `registry.json` listing it
If not	the project has to be the **selected** project. The docs module reads the folders that project cites, not whatever folder happens to be open. A message about there being "no area to put a page in" means the project has no area yet: create a chapter, or start the project from a template

Come back to this step whenever you have more to write. Nothing downstream notices until you build. Writing itself is The editor, later in this guide.

Step 4 — build the site

Build the Site (`temcoDocs.buildSite`), or *Build the Site* on the project page. It builds the project that is **selected**, reading the folders that project cites, so the folder does not have to be open in the window.

> **Screenshot — the Build Center after a build: the four counts, one card per output this project builds, and the newest run at the head of the log.** `media/publishing/04-build.png`

Two things can produce the site, and what is in the folder decides which one runs:

In the project folder	What runs	Where the site lands
a `package.json` with `docs:site` or `docs:build`	that script, after `docs:registry` when the repository has one	wherever the script writes, commonly `dist/site` or docfx's `dist/docs`
neither	the **built-in generator**	`<project>/dist/site`

Either way, the attempt is recorded in the Build Center (`temcoDocs.builds`), and every file the build wrote is recorded against it, so Publish & Releases can list real files rather than hopes.

You should see	`~/temco/projects/t3000/dist/site/index.html` , and a row in the Build Center (`temcoDocs.builds`)
If not	a project made here has no `package.json` , so the built-in generator writes the site, and the log says what it read. If your repository does have `docs:site` or `docs:build` , that script runs instead, and it may write `dist/docs`

The exit code is never the verdict. What counts is a file that did not exist before the run and does now. A generator that exits 0 and writes nothing has failed; one that exits 1 after writing the site has not.

Step 5 — build the project's own outputs (only for a repository)

Skip this step unless T3000 cites a real repository clone under `~/temco/source/` . **Temco Projects: Build** (`temcoProjects.buildProject`) takes the outputs you ticked and runs the repository's own script for each one, **once each**:

> **Screenshot — the terminal during a project build: `docs:registry` , then one script per ticked output, then the list of files it left behind.** `media/publishing/05-project-outputs.png`

Ticked output	Script it looks for, in the repository
Website	`docs:site` , else `docs:build`
Offline help	`build:help:dry`
PDF	`build:help:dry` , else `docs:pdf`
Search index, AI index	nothing: no script here writes them

`docs:registry` runs first whenever the repository has it, because it is what the other scripts read their structure from. The build then records every file that appeared during the run, whatever the exit codes said.

You should see	the terminal run `docs:registry`, then the scripts above, and the log list the files they left behind
If not	the warning names the scripts it looked for. A repository with none of them runs nothing, which is the honest answer. A scaffolded project has no `package.json` at all, so step 4 is its build, and this step is not for it

This is the step that produces a PDF or a `.chm` **for the docs module**: the copies it records and Publish & Releases lists, under their own output. The PDF that the website serves is a different file, made in step 7 from the site's own content.

Step 6 — add the build to the site

Add This Build to the Site (`temcoLearn.addBuild`), or *Choose a build* on the Publishing page. The page that lists every build on disk is `temcoLearn.addBuilds`.

> **Screenshot — the version box, showing "project t3000" and the version field.**
> `media/publishing/06-add-build.png`

It finds every `dist/site` and `dist/docs` two folders below the window's folders, the projects folder and the working folder, and offers the ones that contain an `index.html` or, for a documentation project, an `index.md`. A Markdown build is added as content, so its pages become editable pages of the site; an HTML build is copied beside the catalog as it came. The box then names the project it will land in (`project t3000`) and asks one question: which version. Type `1.0`.

What happens with your answer:

copies	the whole build folder into `<siteRepo>/public/imported/t3000/1.0/`. Adding the same version again **replaces** it
records	`<siteRepo>/content/t3000/site.json`, holding the title and the section. A new project defaults to `documentation`
catalogs	runs the site's own `npm run catalog`, which rewrites `public/catalog.json`, `public/search.json`, `public/content/` and the download bundles
opens	the site page for that project and version, so the result is one click away

You should see	`~/temco/temco-learn/public/imported/t3000/1.0/index.html`, `content/t3000/site.json`, and `t3000` with version `1.0` in `public/catalog.json`
If not	the build must contain an `index.html`. The catalog is rebuilt by this step, so a failure here means the clone has no `npm ci` yet, or has no `catalog` script

The project's id is **its folder's name**, so a build in `<projectsFolder>/t3000/…` is always `t3000`, whatever its pages call themselves. Its section is `documentation` unless `site.json` says otherwise.

Repeat this step every time you rebuild. A build is copied as one version, so a changed site is a *new* version (`1.1`, `2.0`). Re-adding `1.0` replaces what is already there, which is what you want only when you are fixing that same version.

Adding a build is a local act. Nothing is published by this step.

Step 7 — make the downloads

Make the Downloads (`temcoLearn.site.downloads`) is the step between *Add* and *Publish* on the Publishing page.

> **Screenshot — the Publishing page, with the five steps visible and the downloads step running.**
> `media/publishing/07-downloads.png`

It runs the site's own scripts in this order, and stops at the first real failure:

Script	Makes	If it fails
`build:fast`	the catalog, the search index and the bundle the PDFs are printed from	fatal: nothing else can run
`pdf --strict`	one PDF per version, and one per Markdown page, mirrored into `public/pdf/`	fatal, usually because there is no browser: run `npx playwright install chromium`
`chm`	the help files and `public/download/manifest.json`	only a warning: a machine without `hhc.exe` still has a site

You should see	`public/pdf/t3000/1.0.pdf`, `public/download/manifest.json`, and `public/chm/…` only when the help compiler is on the machine
If not	the message names the step that failed: the site build, the PDFs (browser missing), or the help files (a warning, never fatal)

These files are the one thing the deploy cannot make for you. The site's CI has no browser and no `hhc.exe`, so the PDF is printed here and committed, and the deploy **refuses to ship a version whose PDF is missing**. That refusal is the guard you will meet in step 9 if you skip this step.

Come back to this step every time you finish step 6. Each new version needs its own PDF, and the guard counts versions rather than projects.

Step 8 — look at it before publishing

Start the Site (`temcoLearn.site.start`).

> **Screenshot — the site in a browser at localhost:1025: T3000's card under Documentation, and its page open.** `media/publishing/08-preview.png`

It builds the site first if `dist/index.html` is not there, then serves the built pages on one port: `temcoLearn.sitePort`, 1025 by default. What you are looking at is the **clone**, with the catalog that step 6 wrote. It is what a reader will see, minus the address.

Look at	Because
the project's card under its section	the section came from `content/t3000/site.json`, so a wrong section is a one-line fix there
the version selector	every version you added is listed, and `1.0` should be there after step 6
a page's navigation and content	an empty page means the Markdown is empty, not that the build failed
the downloads menu	it lists what step 7 committed, so an empty menu means the manifest is missing
search	this is the site's own index, and a page missing from it has `ship: false`

You should see	`http://localhost:1025/` with T3000 under **Documentation**, and its page opening from the card
If not	the port is `temcoLearn.sitePort`; the site's log in the panel says whether the build or the server is the problem

If a page is missing, it is either unregistered or marked `ship: false`, so go back to step 3 rather than rebuilding. **Stop the Site** (`temcoLearn.site.stop`) when you are finished. The open port, not the process, is the proof that it is running.

Step 9 — publish

Publish (`temcoLearn.publish`) opens the publishing page: what changed, the steps of a publish in order, and the button. Press *Publish* there and confirm the message. **Publish Now** (`temcoLearn.publishNow`) does the same thing from the palette.

> **Screenshot — the Publishing page before the push: the changed paths, the five steps, and the Publish button.** `media/publishing/09-publish.png`

It does four things in order, and it stops at the first one that says no:

1	runs the site's `catalog` , so that what is committed matches what the pages say
2	runs the site's **`guard`** , the same gate its CI runs, and refuses to commit when a version is missing a download
3	runs `git add --all` and makes one commit. The message defaults to `docs: publish the site (<date>)` , and you can replace it
4	runs `git push origin HEAD:main`

Nothing is ever force-pushed. If `main` carries work this clone does not know about, the push is refused, the commit stays local, and a pull followed by another publish fast-forwards it.

Then the site's CI takes over. `pages.yml` runs its checks, builds the site, runs the same guard, and deploys `dist` to GitHub Pages at the address the log printed. A `v*` tag matching `package.json` instead packs the whole site as a zip for handing to somebody.

You should see	in the clone, `git log -1 --stat` shows the commit and `git status` is clean. The log says `pushed <commit>` , the GitHub Actions run on `main` is green, and the address the log printed loads the page a minute later
If not	*the guard refused it* means step 7 was skipped for some version. The log names the missing file, and nothing was committed. *A refused push* means the clone is behind: pull, then publish again

A publish carries everything waiting in the clone, not only the project you were working on. If that is not what you want, open the publishing page first: it lists the paths the commit would take before you press the button.

The editor

You will spend most of your time here, so five minutes learning the surface saves months of hunting for a button.

A page opens in **Temco Editor**, the editor that owns `*.md`. It has one row of controls above the text, the text in the middle, and a **Page** container on the far right. The headings you see in the text are the page's prose; its title in the navigation comes from the registry, so renaming one does not rename the other.

> **Screenshot — a page open in Temco Editor, with the ribbon groups numbered: undo/redo, Style, Lists, marks, blocks, insert, `···`, and the right group.** `media/publishing/10-editor-ribbon.png`

The ribbon, left to right

Group	Holds
undo / redo	`Ctrl+Z`, `Ctrl+Y`. This is the page's own history, not git's
Style	paragraph or heading level, and the alignment of the block
Lists	bullet, numbered and task lists
Marks	bold `Ctrl+B` · italic `Ctrl+I` · underline `Ctrl+U` · strikethrough `Ctrl+Shift+X` · inline code `Ctrl+E` · highlight `Ctrl+Shift+H`
Blocks	quote `Ctrl+Shift+B` · code block `Ctrl+Alt+C` · horizontal rule · table
Insert	**Picture**, **Screenshot**, **Annotate** (only when a picture is selected), **Link to page**
`···`	everything the row has no space for, including *Open the source* (the plain Markdown, for a paste that must not be reformatted) and *Page properties*
right group	Find `Ctrl+F` · Outline `Ctrl+Alt+O` · **Read the page**, which toggles to *Edit the page* · Preview `Ctrl+Alt+P` · Diff `Ctrl+Alt+D` · Source · Page properties

The same actions are in the editor title bar and in the right-click menu under **Temco Markdown**, so nothing is reachable only from the ribbon.

Read the page is the one people miss. It turns the editor into the rendered page, links and all, without opening Preview. Toggle it back when you want to edit again.

Pictures

Click a picture once to select it. A ring appears, with drag handles at its edges. Drag a handle to resize, and the width is written into the Markdown title as `"w=480"`, so the built site and the offline help honour it too.

> **Screenshot — a picture selected in the editor, with the drag handles and the 25% · 50% · 75% · 100% · Delete menu visible.** `media/publishing/11-editor-picture.png`

With a picture selected, the inline menu offers the widths **25%** · **50%** · **75%** · **100%** and **Delete**. `Ctrl+Z` puts a resize back. Nothing is written to the file until the editor saves, so an experiment costs you nothing until you decide to keep it.

Screenshot inserts a capture straight into the page's `media/` folder. A capture taken with no page open waits in **Needs You**, where *Attach the Screenshot to a Page…* files it later. **Annotate** draws on the selected picture, and one arrow on a screenshot is often worth a paragraph of text.

The Page container, on the right

> **Screenshot — the Page container: the page's Outline, and the Draft the AI wrote beside the page's own text.** `media/publishing/12-editor-sidebar.png`

Two things live there. The **Outline** lists the page's headings, and clicking one jumps to it. The **Draft** appears when the AI module has written one. A draft is never applied by itself: you accept it, and it becomes a single undoable edit.

Preview, Diff and Source

> **Screenshot — Preview of a page beside the editor, then the Diff view against HEAD.** `media/publishing/13-editor-preview.png`

Control	Shows	Use it when
Preview `Ctrl+Alt+P`	the page as the build renders it, in a tab of its own	you are checking a table, a callout or a picture width
Diff with HEAD `Ctrl+Alt+D`	what changed since the last commit	before publishing, especially on someone else's page
Source	the raw Markdown	a paste keeps coming out wrong
Page properties	a **read-only sheet** of the page's registry entry: title, area, chapter, order, `ship` and audience, plus the two ways to change it (the registry, or the page)	a page does not appear where you expect

Keyboard, all in one place

Shortcut	Does
`Ctrl+B` / `Ctrl+I` / `Ctrl+U`	bold / italic / underline
`Ctrl+Shift+X`	strikethrough
`Ctrl+E`	inline code
`Ctrl+Shift+H`	highlight
`Ctrl+Shift+B` / `Ctrl+Alt+C`	block quote / code block
`Ctrl+F`	find in the page
`Ctrl+Alt+O`	outline
`Ctrl+Alt+P` / `Ctrl+Alt+D`	preview / diff with HEAD
`Ctrl+Z` / `Ctrl+Y`	undo / redo
`Ctrl+S`	save the page to disk

Running it again

After the first publish, the loop is shorter than the guide above, because the folders, the project and the site clone already exist. What you re-run depends on what changed:

What changed	Build	Add	Downloads	Publish
pages only, same version	✓	✓ (same version — it replaces)	✓	✓
pages, and this is a new release	✓	✓ (new version)	✓	✓
nothing — you only want to push someone else's changes	—	—	—	✓
the site's own code or content	—	—	—	✓

The four commands, in order:

```
Temco Docs:    Build the Site                 temcoDocs.buildSite
Temco Learn:   Add This Build to the Site     temcoLearn.addBuild
Temco Learn:   Make the Downloads             temcoLearn.site.downloads
Temco Learn:   Publish                        temcoLearn.publish
```

Two habits save the most time. **Add a new version for a new release**, because re-adding the same version replaces it and readers lose the old one. And **redo the downloads every time you add a version**, because the guard counts versions, so a new version without its PDF blocks the publish rather than the deploy.

The same flow, as files: T3000

```
~/temco/projects/t3000/                  the project, and only the project
├── docs/help/registry.json              the page registry
├── docs/commissioning/*.md              your pages
└── dist/site/index.html                 step 4: the site build

~/temco/temco-learn/                     the site clone
├── content/t3000/site.json              step 6: the title and the section
├── public/imported/t3000/1.0/index.html step 6: the build's own pages
├── public/pdf/t3000/1.0.pdf             step 7: the whole-guide PDF
├── public/chm/t3000/1.0.chm             step 7: the help file, when it compiles
├── public/download/manifest.json        step 7: what the download menu offers
├── public/catalog.json                  what the site reads, rewritten by steps 6 and
└── dist/                                step 9: what CI deploys
```

What the files look like

Three files carry the whole arrangement. Knowing their shape lets you tell a wrong page from a wrong build without guessing.

`~/temco/projects/t3000/docs/help/registry.json` is written by the create flow and by every page you add. It is the only list of pages; nothing else decides what exists.

JSON

```
{
  "version": 1,
  "areas": { "t3000": { "title": "t3000", "audience": "user", "noindex": false } },
  "pages": {
    "t3000/commissioning/ports": {
      "title": "Ports and wiring",
      "area": "t3000", "chapter": "commissioning", "order": 10,
      "audience": "user", "ship": true, "owner": "docs"
    }
  }
}
```

`ship: false` leaves a page out of the built site, and `order` decides its place inside the chapter. The key (`t3000/commissioning/ports`) must equal the file's path under `docs/`, or the page counts as missing.

`~/temco/temco-learn/content/t3000/site.json` is written once by step 6 and kept after that. It is how the site names the project and files it under a section.

JSON

```
{ "title": "T3000", "section": "documentation" }
```

Add `"summary"` for the sentence shown on the project's card, and change `"section"` to move the whole project to another category.

`~/temco/temco-learn/public/catalog.json` is generated and never edited. Every row is either a Markdown page the site owns or an imported page your build wrote, in one reading order.

JSON

```
{ "projects": [ { "id": "t3000", "title": "T3000", "section": "documentation",
                  "versions": [ { "version": "1.0",
                                  "pages": [ { "href": "index.html", "title": "T3000", "
```

That `"kind": "build"` is the giveaway: the page came from your build rather than from the site's own content. If it is missing after step 6, the catalog did not run.

Reference: what the outputs mean

The ticks on a project say what it is meant to produce, and one build is meant to produce all of them.

Output	Is	Which script runs, in your repository
Website	static pages for Temco Learn	`docs:site`, else `docs:build`, else the built-in generator
PDF	one document in a printed layout	`build:help:dry`, else `docs:pdf`
Offline help	the `.chm`	`build:help:dry`
Search index	the index the site loads	none here: the site builds it with `npm run search`
AI index	embeddings, kept on this machine only	none here: nothing is uploaded

A build writes inside your project folder. The site goes to `dist/site` (or `dist/docs`, when a repository's own generator wrote it), and the PDF and the `.chm` go wherever your scripts put them, commonly `dist/` or a `HelpDocs` folder beside the repository. Every file a build leaves behind is recorded against it, so the Builds and Downloads pages can list it.

Two things worth doing on purpose

A guard nobody has watched refuse is a guard nobody can trust.

Try this	You should see
Delete `public/pdf/t3000/1.0.pdf`, then publish	the publish **stops before committing** and names the missing file. Run *Make the Downloads* and publish again, and it goes through
Rename the project's page title, rebuild, then add the build again as `1.1`	version `1.1` lands under `t3000`: `content/` still holds one folder, and the catalog lists one project with two versions

When a step goes wrong

Step	What you see	What it usually is	Where to look
4	the build is not in the list to add	there is no `index.html` or `index.md` in it, or it sits more than two folders below a root	the folder itself. The roots are the window's folders, the projects folder and the working folder
6	adding refuses	the site clone is not where `temcoLearn.siteRepo` says	that setting, and whether the folder has `package.json` and `content/sections.json`
6, 9	the catalog did not run	there is no `npm ci` in the clone, or no `catalog` script	the site log
9	publishing refuses	the site's guard: a version has no committed PDF	the guard lines in the log
9	publishing pushes, but the site looks unchanged	the commit carried nothing, because the clone matched the last publish	`git log` and `git status` in the clone
6	the site page is wrong after a rebuild	the build was rebuilt but never added again as a new version	`public/catalog.json` and `public/imported/`

Beyond the flow

A project can do more than publish. None of the following is needed to get a page live, and all of it is available once you have one.

Bringing in material that already exists

A **source** is where pages are written from: a repository, a folder, or a document. There are two doors to the same result: the project's own page (`temcoProjects.editProject`), or the `Pages` view title → **Add a Source** (`temcoProjects.addSource` , the plug beside `+`).

If you already have a `docs/` folder, create the project with **From existing files** and choose that folder or file. A folder that already has `docs/help/registry.json` is cited as it stands, and nothing is copied. Anything else (Markdown, HTML, Word `.docx` , or a compiled `.chm`) is converted **once** into `docs/` and a registry, so from then on it is ordinary pages. A source is recorded once and can be cited by more than one project, and nothing is ever written back to it.

Checking before you build

Validate (`temcoDocs.validate`) answers one question: would a reader find a broken link, a missing page, or a page nobody registered? It reports in the Problems panel. A build does not fail on what it finds; the published site simply contains it. So run it before step 4.

Releasing a build locally

Publish on the Publish & Releases page (`temcoDocs.publishRelease`) is not publishing to the site. It copies the last build's artifacts into the application's own storage, at `published/<project>/<version>` with a manifest, behind a confirmation that names the version, the destination and the file count. `temcoDocs.openPublished` opens that copy, `temcoDocs.servePublished` serves it and `temcoDocs.stopServing` stops it. Use it to hand a build to somebody, or to keep a version that will not change. Nothing is pushed anywhere.

Exporting

Export… (`temcoDocs.export`) opens the Exports page, which tells you what you get before you get it. Every export writes into the project's `dist/` , and the notification offers *Show in Folder*.

> **Screenshot — the Exports page: the four shapes, each saying what is inside it and where it lands.**
> `media/publishing/14-exports.png`

Shape	Command	You get
the site, as a zip	`temcoDocs.exportSite`	`index.html` , `assets/site.css` , every page and each page's `media/` : the finished website before it is served from anywhere
one page, as one file	`temcoDocs.exportPage`	a single `.html` with the stylesheet inlined and every picture a `data:` URI, so it opens on a machine that has never seen the project
one page, as a Word document	`temcoDocs.exportDocx`	a `.docx` with headings, lists, tables and the pictures embedded: the format reviewers ask for
the pages, as a Markdown bundle	`temcoDocs.exportMarkdown`	every `.md` , each folder's `media/` and `registry.json` : the sources, for another tool or for an archive

Nothing is uploaded. A page with very large pictures makes a very large single file, so export the site as a zip when the pictures matter more than the convenience.

Translating a page

Translate This Page... (`temcoDocs.translatePage`) writes the translation **beside** the page, never over it: `docs/guide/setup.md` becomes `docs/guide/setup.de.md` .

> **Screenshot — the language picker, and the translated sibling page open beside the original.**
> `media/publishing/15-translate.png`

- Only the prose is translated. Code fences, inline code, table rules, link targets, image paths, front matter and the leading `#` , `-` and `>` markers are preserved byte for byte, so a translated page still builds, still links and still shows its pictures.
- **Once**: an Azure AI Translator resource. Its key goes into the OS keychain with **Set the Translator Key** (`temcoDocs.setTranslatorKey`), and the endpoint and region are the settings `temcoDocs.translator.endpoint` and `temcoDocs.translator.region` .
- **One request per page**, sent only when you run the command. Nothing is translated in the background.
- **Errors**: `401` means a key that was refused, `403` a key without a Translator resource (check the region), and `429` the free tier's rate limit. A page that is all code only says there is no prose, and costs nothing. Translating again asks before replacing the sibling file.
- A translation is an ordinary page. The registry and the review flow treat it like any other file, and a build includes it. Listing it in a language menu on the site is a registry decision, not the translator's.

When something else goes wrong

The table above covers the publishing pipeline. These are the rest:

What you see	What it means	What to do
Build the Site says *no `docs:site` (or `docs:build`) script*	the project has a `package.json` but no site script	add one, or delete the `package.json` so the built-in generator takes over
Build the Site says *has no pages marked for publishing*	no page is registered for the site, or every registered page is missing its file	open `Pages`, look for `missing` rows, and use **Create the Missing Page File**
the site builds but a page is empty	the page is registered and its file exists, but the Markdown is empty	write it, or **Retire the Page**
a page is missing from the published site	it is not registered, or it is not marked for the site	`Pages` → `…` → **Open Registry**
an export says *has no pages to export*	no registered page has a file yet	create or generate the pages first
a single-file page shows a broken picture	the picture's path in the Markdown is wrong, or it is not under the page's `media/`	fix the reference, because the same link is broken on the site
the editor shows raw Markdown	the file opened as text	in `Pages`, right-click the page → **Open the Page**; or in Explorer, **Reopen Editor With…** → **Temco Editor**
a picture is full width on the web but small in the editor	the page is older than the width support	reselect the picture, drag a handle, save
The Temco Editor surface is not built — `markdown.js`, `markdown.css` are missing	the window is running compiled extension-host code whose webview bundle was never built, or was removed by a build that cleans `out/`	`npm run compile-temco` from the repository root, then **Developer: Reload Window**. A plain `npm run compile` stops before the webviews when an earlier check fails, so the application compiles and its pages still have no surface

The output explains itself: **View** → **Output** → **`Temco Docs`**. Every build, publish, export and generator decision is written there, with the path it used.

Appendix — every button, one page

Every control the platform puts on screen, grouped by where it lives. The command id beside each one runs the same action from the Command Palette (`Ctrl+Shift+P`). Every id here was checked against the manifests, so if a row is in this table, the command exists.

Projects (activity bar)

Button	Where	Command
Create a Project	`Projects` view title `+`	`temcoProjects.newProject`
Open the Project Page	`Projects` view title	`temcoProjects.openProjectPage`
Remove All Projects	`Projects` view title	`temcoProjects.removeAllProjects`
Start (open a project)	right-click a project row	`temcoProjects.openProject`
Edit (sources of a project)	right-click a project row	`temcoProjects.editProject`
Project Detail	right-click a project row	`temcoDocs.projectPage`
Build (the project's own build)	right-click a project row	`temcoProjects.buildProject`
Stop Working on a Project	right-click the *current* row	`temcoProjects.clearProject`

Documentation (activity bar)

Button	Where	Command
New Page	`Pages` title ＋	`temcoDocs.newPage`
New Folder…	`Pages` title 📁	`temcoDocs.newChapter`
Add a Source	`Pages` title 🔌	`temcoProjects.addSource`
Capture a Screenshot	`Pages` title 📷	`temcoDocs.captureImage`
Paste Screenshot	`Pages` title 📋	`temcoDocs.pasteImage`
Refresh	every view title ⟳	`temcoDocs.refresh`
Open the Page	`Pages` … / page row	`temcoDocs.openPage`
Validate	`Pages` …	`temcoDocs.validate`
Overview	`Pages` …	`temcoDocs.overview`
Open Registry	`Pages` …	`temcoDocs.openRegistry`
Import a Word Document	`Pages` …	`temcoDocs.importDocx`
Create the Missing Page File	right-click a `missing` row	`temcoDocs.createPage`
Rename the Page Title / Move Up / Move Down / Retire	right-click a page	`temcoDocs.renamePage`, `.movePageUp`, `.movePageDown`, `.retirePage`
Attach the Screenshot to a Page…	right-click a capture in `Needs You`	`temcoDocs.attachCapture`
Build the Site	`Publish` title 🌐	`temcoDocs.buildSite`
Publish to GitHub Pages	`Publish` title ☁	`temcoDocs.publishSite`
Build the Offline Help	`Publish` title 📦	`temcoDocs.buildHelp`
Export…	Publish & Releases (opens the Exports page)	`temcoDocs.export`
Export the Site as a Zip	palette	`temcoDocs.exportSite`
Export the Page as HTML	palette, page right-click	`temcoDocs.exportPage`

Button	Where	Command
Export the Pages as a Markdown Bundle	palette	`temcoDocs.exportMarkdown`
Export the Page as Word	Exports page, palette	`temcoDocs.exportDocx`
Review Center (row)	`Publish` view	`temcoDocs.review` → opens the Review Center page
Build Center (row)	`Publish` view	`temcoDocs.builds` → opens the Build Center page
Publish & Releases (row)	`Publish` view	`temcoDocs.releases` → opens the Publish & Releases page

Editor (while a page is open)

Button	Where	Command
Bold / Italic / Link to a Page / Format…	editor title	`temcoDocs.format.bold`, `.italic`, `.linkPage`, `.pick`
Preview the Page	editor title, `Ctrl+Alt+P`	`temcoDocs.previewPage`
Diff the Page with HEAD	editor title, `Ctrl+Alt+D`	`temcoDocs.diffPage`
Page Properties	editor title	`temcoDocs.pageProperties`
Capture / Paste Screenshot	editor title	`temcoDocs.captureImage`, `temcoDocs.pasteImage`
Open the source (plain text)	editor title, ribbon `...`	`temcoEditor.openSource`
Temco Markdown (whole menu)	editor right-click	submenu with every format verb

The pages themselves

Button	On the page	Command
Build Site	Build Center, Publish & Releases	`temcoDocs.buildSite`
Build the Website / PDF / … (one card per output)	Build Center	`temcoDocs.buildOutput`
Build Offline Help	Build Center, Publish & Releases	`temcoDocs.buildHelp`
Refresh	Review Center	`temcoDocs.review`
Open folder	Build Center, Publish & Releases	`temcoDocs.revealArtifactFolder`
Open (a file)	Publish & Releases → a build	`temcoDocs.openArtifact`
Publish	Publish & Releases	`temcoDocs.publishRelease`
Publish to GitHub Pages	Publish & Releases	`temcoDocs.publishSite`
Export…	Publish & Releases	`temcoDocs.export`
Serve / Serve the Newest	Publish & Releases, release rows	`temcoDocs.servePublished`
Open (a release)	Publish & Releases → release rows	`temcoDocs.openPublished`
Stop Serving the Site	palette	`temcoDocs.stopServing`

The site

Button	On the page	Command
Start the Site	Documentation Site	`temcoLearn.site.start`
Stop the Site	Documentation Site	`temcoLearn.site.stop`
Build the Site (the site itself)	Documentation Site	`temcoLearn.site.build`
Make the Downloads	Publishing, Documentation Site	`temcoLearn.site.downloads`
Add a Build to the Site	Publishing	`temcoLearn.addBuilds`
Publish	Publishing	`temcoLearn.publish`
Publish Now	Publishing	`temcoLearn.publishNow`

Settings

Button	On the page	Command
General	Settings	`temcoWorkbench.settings.general`
Modules	Settings	`temcoWorkbench.settings.modules`
AI Runtime	Settings	`temcoWorkbench.settings.ai`
Branding	Settings	`temcoWorkbench.settings.branding`
Publishing	Settings	`temcoWorkbench.settings.publishing`
Repositories	Settings	`temcoRepositories.settings`
Check the Platform for Updates	Modules	`temcoWorkbench.settings.checkUpdates`
Test the Publish Target	Publishing	`temcoWorkbench.settings.testPublish`